

Python Mastery: Flask Web Framework

Building Lightweight Web Applications with Python

1. What is Flask?

Flask is a micro-framework for Python based on Werkzeug and Jinja2. It is called a "micro" framework because it does not require particular tools or libraries. It has no database abstraction layer, form validation, or any other components where pre-existing third-party libraries provide common functions.

A Minimal Application

A minimal Flask application looks something like this:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return '<p>Hello, World!</p>'

if __name__ == '__main__':
    app.run(debug=True)
```

2. Routing

Modern web applications use meaningful URLs to help users. Flask uses the `route()` decorator to bind a function to a URL.

```

@app.route('/hello')
def hello():
    return 'Hello, Guest'

@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return f'User {username}'

```

3. HTTP Methods

By default, a route only answers to **GET** requests. You can use the `methods` argument of the `route()` decorator to handle different HTTP methods.

```

from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()

```

4. Templates

Generating HTML from within Python is cumbersome. Flask leverages the **Jinja2** template engine to render HTML files separately.

```

from flask import render_template

@app.route('/hello/')
@app.route('/hello/<name>')
def hello_name(name=None):
    return render_template('hello.html', name=name)

```

5. Static Files

Dynamic web applications also need static files. That's usually where the **CSS** and **JavaScript** files are coming from. Just create a folder called `static` in your package or next to your module and it will be available at `/static` on the application.

